

Министерство образования и науки Российской Федерации
федеральное государственное автономное образовательное учреждение высшего образования

**“ НАЦИОНАЛЬНЫЙ ИССЛЕДОВАТЕЛЬСКИЙ
УНИВЕРСИТЕТ ИТМО”**

Факультет прикладной информатики (фПИИ)

Направление подготовки (специальность) – 45.03.04 (Языковые модели и
Искусственный Интеллект)

Алгоритмы и структуры данных

Дополнительные задачи

Вариант 21

Выполнил студент: Попов Глеб Ильич Группа № К3260

Преподаватель: Харлунин Александр Александрович

г. Санкт-Петербург 2026 г

Задача №4. Сбор подписей

Текст задачи:

Вы несете ответственность за сбор подписей всех жильцов определенного здания. Для каждого жильца известно время, когда он находится дома.

Дан набор отрезков на прямой. Нужно выбрать минимальное количество точек так, чтобы каждый отрезок содержал хотя бы одну выбранную точку.

То есть необходимо посетить здание минимальное количество раз так, чтобы попасть хотя бы в один момент времени из промежутка каждого жильца.

Формат ввода:

В первой строке входного файла `input.txt` содержится число n - количество отрезков.

В следующих n строках содержатся два целых числа a_i и b_i - координаты начала и конца i -го отрезка.

Ограничения:

$$1 \leq n \leq 100$$

$$0 \leq a_i, b_i \leq 10^9$$

Все числа целые.

Формат вывода:

В первой строке выходного файла `output.txt` нужно вывести минимальное количество точек m .

Во второй строке нужно вывести координаты выбранных точек.

Если существует несколько правильных ответов, можно вывести любой.

Код:

```
import time
import tracemalloc

def solve():
    with open("input.txt", "r", encoding="utf-8") as f:
        data = list(map(int, f.read().split()))
```

```

n = data[0]

segments = []
pos = 1

for _ in range(n):
    left = data[pos]
    right = data[pos + 1]
    pos += 2

    segments.append((left, right))

segments.sort(key=lambda x: x[1])

points = []
current_point = None

for left, right in segments:
    if current_point is None or current_point < left:
        current_point = right
        points.append(current_point)

with open("output.txt", "w", encoding="utf-8") as f:
    f.write(str(len(points)) + "\n")
    if points:
        f.write(" ".join(map(str, points)) + "\n")
    else:
        f.write("\n")

tracemalloc.start()
start_time = time.perf_counter()

solve()

finish_time = time.perf_counter()
cur, peak = tracemalloc.get_traced_memory()
tracemalloc.stop()

with open("stats.txt", "w", encoding="utf-8") as f:
    f.write(f"time = {finish_time - start_time:.6f} sec\n")
    f.write(f"memory = {peak / 1024:.2f} KB\n")

```

Текстовое объяснение решения:

В этой задаче необходимо выбрать минимальное количество точек, которые покрывают все заданные отрезки.

Для этого используется жадный алгоритм. Сначала все отрезки сортируются по правой границе. Это нужно для того, чтобы сначала рассматривать тот отрезок, который заканчивается раньше остальных.

После сортировки программа проходит по всем отрезкам. Если текущий отрезок уже покрыт последней выбранной точкой, то ничего добавлять не нужно. Если же последняя выбранная точка не входит в текущий отрезок, то выбирается новая точка - правая граница текущего отрезка.

Такой выбор является оптимальным, потому что правая граница текущего отрезка точно покрывает этот отрезок и при этом находится как можно правее. Значит, она может покрыть больше следующих отрезков.

После обработки всех отрезков программа выводит количество выбранных точек и сами точки.

Таблица тестирования:

время	память	input	output
0.002921 sec	10.38 KB	3 1 3 2 5 3 6	1 3
0.001712 sec	10.44 KB	4 4 7 1 3 2 5 5 6	2 3 6
0.001538 sec	10.38 KB	3 1 2 3 4 5 6	3 2 4 6
0.002149 sec	10.44 KB	4	3

		1 10 2 3 4 5 6 7	3 5 7
--	--	---------------------------	-------

Вывод по задаче

В ходе решения задачи был реализован жадный алгоритм покрытия отрезков минимальным количеством точек. Для этого отрезки сортируются по правой границе, после чего для каждого еще не покрытого отрезка выбирается его правый конец. Программа корректно находит минимальное количество точек и выводит один из возможных оптимальных наборов.

Задача №5. Максимальное количество призов

Текст задачи:

Вы организуете конкурс для детей. В качестве призового фонда есть n конфет.

Нужно раздать конфеты как можно большему количеству победителей так, чтобы все призы были различными по количеству конфет.

То есть необходимо представить число n в виде суммы максимального количества попарно различных натуральных чисел.

Например, число 6 можно представить как:

$$1 + 2 + 3$$

В этом случае получится выдать призы трем участникам.

Формат ввода:

Во входном файле input.txt дано одно целое число n .

Ограничения:

$$1 \leq n \leq 10^9$$

Формат вывода:

В первой строке выходного файла output.txt нужно вывести максимальное количество различных слагаемых k .

Во второй строке нужно вывести сами слагаемые.

Если существует несколько правильных ответов, можно вывести любой.

Код:

```
import time
import tracemalloc

def solve():
    with open("input.txt", "r", encoding="utf-8") as f:
        n = int(f.read().strip())

    prizes = []
    current = 1

    while n > 0:
        if n - current > current:
            prizes.append(current)
            n -= current
            current += 1
        else:
            prizes.append(n)
            break

    with open("output.txt", "w", encoding="utf-8") as f:
        f.write(str(len(prizes)) + "\n")
        f.write(" ".join(map(str, prizes)) + "\n")

tracemalloc.start()
start_time = time.perf_counter()

solve()

finish_time = time.perf_counter()
cur, peak = tracemalloc.get_traced_memory()
tracemalloc.stop()

with open("stats.txt", "w", encoding="utf-8") as f:
```

```
f.write(f"time = {finish_time - start_time:.6f} sec\n")
f.write(f"memory = {peak / 1024:.2f} KB\n")
```

Текстовое объяснение решения:

В этой задаче нужно представить число n в виде суммы максимального количества различных натуральных чисел.

Для этого используется жадный алгоритм. Сначала в ответ добавляется минимальное возможное число 1, затем 2, затем 3 и так далее. Такой подход позволяет использовать как можно больше маленьких различных чисел, а значит увеличить количество слагаемых.

На каждом шаге программа проверяет, можно ли взять текущее число `current`. Если после его вычитания остаток будет больше `current`, значит остаток можно будет использовать для следующего большего слагаемого, и число добавляется в ответ.

Если же остаток после вычитания оказался бы меньше или равен `current`, то брать `current` уже нельзя, потому что тогда последнее слагаемое могло бы совпасть с одним из уже выбранных или нарушить порядок различия. В этом случае весь оставшийся остаток добавляется последним числом.

После этого программа выводит количество слагаемых и сами слагаемые.

Таблица тестирования:

время	память	input	output
0.001612 sec	10.23 KB	6	3 1 2 3
0.001420 sec	10.23 KB	8	3 1 2 5

0.001436 sec	10.33 KB	2	1 2
0.001940 sec	10.26 KB	15	5 1 2 3 4 5

Вывод по задаче

В ходе решения задачи был реализован жадный алгоритм разложения числа на максимальное количество различных натуральных слагаемых. Программа последовательно выбирает минимальные возможные числа, а оставшийся остаток добавляет последним слагаемым. Такой подход позволяет получить максимальное количество призов и корректно работает при больших значениях n .

Задача №6. Максимальная зарплата

Текст задачи:

В качестве последнего вопроса успешного собеседования начальник дает несколько листков бумаги с числами и просит составить из этих чисел наибольшее возможное число.

Нужно расположить данные числа в таком порядке, чтобы при их склеивании получилось максимальное число.

Например, если даны числа 21 и 2, то лучше поставить сначала 2, а потом 21, потому что число 221 больше, чем 212.

Формат ввода:

В первой строке входного файла input.txt содержится число n — количество чисел.

Во второй строке содержатся числа $a_1, a_2, \dots, a_n$.

Ограничения:

$$1 \leq n \leq 100$$

$$1 \leq a_i \leq 1000$$

Формат вывода:

В выходной файл output.txt нужно вывести наибольшее число, которое можно составить из данных чисел..

Код:

```
import time
import tracemalloc
from functools import cmp_to_key

def compare(a, b):
    if a + b > b + a:
        return -1
    if a + b < b + a:
        return 1
    return 0

def solve():
    with open("input.txt", "r", encoding="utf-8") as f:
        data = f.read().split()

    n = int(data[0])
    numbers = data[1:1 + n]

    numbers.sort(key=cmp_to_key(compare))

    answer = "".join(numbers)

    with open("output.txt", "w", encoding="utf-8") as f:
        f.write(answer + "\n")

tracemalloc.start()
start_time = time.perf_counter()

solve()

finish_time = time.perf_counter()
cur, peak = tracemalloc.get_traced_memory()
tracemalloc.stop()

with open("stats.txt", "w", encoding="utf-8") as f:
    f.write(f"time = {finish_time - start_time:.6f} sec\n")
```

```
f.write(f"memory = {peak / 1024:.2f} KB\n")
```

Текстовое объяснение решения:

В этой задаче нужно расположить числа так, чтобы после их объединения получилось максимально возможное число.

Обычная сортировка по значению здесь не подходит. Например, число 23 больше, чем 3 как отдельное число, но при составлении максимального числа порядок должен быть 3 23, потому что 323 больше, чем 233.

Поэтому для двух чисел a и b сравниваются две строки: $a + b$ и $b + a$. Если $a + b$ больше, значит число a должно стоять раньше b . Если больше $b + a$, то раньше должно стоять число b .

После такой сортировки все числа объединяются в одну строку.

Полученная строка и является максимальным числом, которое можно составить из данных чисел.

Таблица тестирования:

время	память	input	output
0.001425 sec	10.27 KB	2 21 2	221
0.001304 sec	10.44 KB	3 23 39 92	923923
0.002272 sec	10.33 KB	5 9 4 6 1 9	99641
0.002246 sec	10.40 KB	3 12 121 9	912121

Вывод по задаче

В ходе решения задачи был реализован алгоритм составления наибольшего числа из набора целых чисел. Для правильного порядка используется специальное сравнение двух чисел по результатам их склеивания в двух возможных вариантах. Такой подход позволяет корректно обрабатывать как однозначные, так и многозначные числа.

Задача №7. Опознавание двоичного дерева поиска (усложненная версия)

Текст задачи:

Дано двоичное дерево с ключами, которые могут повторяться. Нужно проверить, является ли это дерево корректным двоичным деревом поиска.

Для каждой вершины должны выполняться условия:

все ключи в левом поддереве строго меньше ключа вершины;

все ключи в правом поддереве больше или равны ключу вершины.

То есть одинаковые ключи разрешены, но дубликаты должны находиться только в правом поддереве.

Если дерево является корректным двоичным деревом поиска, нужно вывести CORRECT, иначе - INCORRECT.

Формат ввода:

В первой строке входного файла input.txt содержится число n - количество вершин дерева.

Следующие n строк содержат описание вершин. Каждая строка содержит три целых числа:

Ki Li Ri

где K_i — ключ вершины, L_i — индекс левого ребенка, R_i — индекс правого ребенка.

Если у вершины нет левого или правого ребенка, вместо индекса указано -1.

Вершины нумеруются от 0 до $n - 1$. Корнем дерева является вершина 0.

Ограничения:

$$0 \leq n \leq 100000$$

$$-2^{31} \leq K_i \leq 2^{31} - 1$$

$$-1 \leq L_i, R_i \leq n - 1$$

Дерево не обязательно сбалансировано.

Формат вывода:

В выходной файл output.txt нужно вывести:

CORRECT, если дерево является корректным двоичным деревом поиска.

INCORRECT, если дерево не является корректным двоичным деревом поиска..

Код:

```
import time
import tracemalloc

def solve():
    with open("input.txt", "r", encoding="utf-8") as f:
        data = f.read().split()

    n = int(data[0])

    if n == 0:
        with open("output.txt", "w", encoding="utf-8") as f:
            f.write("CORRECT\n")
```

```

        return

    key = [0] * n
    left = [0] * n
    right = [0] * n

    pos = 1

    for i in range(n):
        key[i] = int(data[pos])
        left[i] = int(data[pos + 1])
        right[i] = int(data[pos + 2])
        pos += 3

    stack = [(0, None, None)]

    while stack:
        v, min_value, max_value = stack.pop()

        if min_value is not None and key[v] < min_value:
            with open("output.txt", "w", encoding="utf-8") as f:
                f.write("INCORRECT\n")
            return

        if max_value is not None and key[v] >= max_value:
            with open("output.txt", "w", encoding="utf-8") as f:
                f.write("INCORRECT\n")
            return

        if left[v] != -1:
            stack.append((left[v], min_value, key[v]))

        if right[v] != -1:
            stack.append((right[v], key[v], max_value))

    with open("output.txt", "w", encoding="utf-8") as f:
        f.write("CORRECT\n")

tracemalloc.start()
start_time = time.perf_counter()

solve()

finish_time = time.perf_counter()
cur, peak = tracemalloc.get_traced_memory()
tracemalloc.stop()

```

```
with open("stats.txt", "w", encoding="utf-8") as f:
    f.write(f"time = {finish_time - start_time:.6f} sec\n")
    f.write(f"memory = {peak / 1024:.2f} KB\n")
```

Текстовое объяснение решения:

В этой задаче нужно проверить корректность двоичного дерева поиска с учетом того, что одинаковые ключи разрешены только в правом поддереве.

Недостаточно просто сравнивать каждую вершину с ее детьми. Нужно учитывать ограничения от всех предков. Например, вершина может быть меньше своего родителя, но при этом нарушать ограничение от более высокой вершины.

Поэтому для каждой вершины во время обхода хранятся допустимые границы значений:

`min_value` - минимально допустимое значение;

`max_value` - максимально допустимое значение.

Для левого ребенка верхняя граница становится равной ключу текущей вершины, потому что все элементы слева должны быть строго меньше.

Для правого ребенка нижняя граница становится равной ключу текущей вершины, потому что справа разрешены элементы больше или равные текущему ключу.

Обход реализован через стек, без рекурсии. Это важно, потому что дерево может быть очень глубоким, и рекурсивное решение может привести к переполнению стека.

Если во время обхода находится вершина, которая не попадает в допустимый диапазон, программа сразу выводит INCORRECT. Если все вершины прошли проверку, выводится CORRECT

Таблица тестирования:

время	память	input	output
0.002356 sec	10.53 KB	3 2 1 2 1 -1 -1 3 -1 -1	CORRECT
0.001352 sec	10.56 KB	3 2 1 2 1 -1 -1 2 -1 -1	CORRECT
0.002128 sec	10.52 KB	3 2 1 2 2 -1 -1 3 -1 -1	INCORRECT
0.001432 sec	10.21 KB	0	CORRECT

Вывод по задаче

В ходе решения задачи была реализована проверка корректности двоичного дерева поиска с повторяющимися ключами. Для каждой вершины хранятся допустимые границы значений, что позволяет учитывать ограничения не только от родителя, но и от всех предков. Решение выполнено без рекурсии, поэтому корректно работает даже на глубоких деревьях.

Задача №9. Удаление поддеревьев

Текст задачи:

Дано некоторое двоичное дерево поиска. Также даны запросы на удаление из него вершин с заданными ключами.

При удалении вершина удаляется целиком вместе со своим поддеревом.

После каждого запроса нужно вывести количество вершин, которые остались в дереве.

Если вершины с заданным ключом нет или она уже была удалена вместе с каким-то поддеревом, дерево не изменяется.

Формат ввода:

В первой строке входного файла `input.txt` находится число N — количество вершин в дереве.

В следующих N строках находится описание вершин дерева. Каждая строка содержит три числа:

K_i L_i R_i

где K_i - ключ вершины, L_i - номер левого ребенка, R_i - номер правого ребенка.

Если ребенка нет, вместо его номера указано 0.

После описания дерева находится число M - количество запросов на удаление.

В следующей строке находятся M чисел - ключи вершин, которые нужно удалить вместе с их поддеревьями.

Ограничения:

$$1 \leq N \leq 2 * 10^5$$

$$|K_i| \leq 10^9$$

$$1 \leq M \leq 2 * 10^5$$

Все ключи в дереве различны.

Гарантируется, что дерево является двоичным деревом поиска.

Гарантируется, что корень дерева никогда не будет удален.

Высота дерева не превосходит 25.

Формат вывода:

В выходной файл output.txt нужно вывести М строк.

В каждой строке нужно вывести количество вершин, оставшихся в дереве после очередного запроса.

Код:

```
import time
import tracemalloc

def solve():
    with open("input.txt", "r", encoding="utf-8") as f:
        data = f.read().split()

    n = int(data[0])

    key = [0] * (n + 1)
    left = [0] * (n + 1)
    right = [0] * (n + 1)
    parent = [0] * (n + 1)

    key_to_vertex = {}

    pos = 1

    for i in range(1, n + 1):
        key[i] = int(data[pos])
        left[i] = int(data[pos + 1])
        right[i] = int(data[pos + 2])
        pos += 3

        key_to_vertex[key[i]] = i

        if left[i] != 0:
            parent[left[i]] = i
```

```

        if right[i] != 0:
            parent[right[i]] = i

subtree_size = [0] * (n + 1)

for i in range(n, 0, -1):
    subtree_size[i] = 1

    if left[i] != 0:
        subtree_size[i] += subtree_size[left[i]]

    if right[i] != 0:
        subtree_size[i] += subtree_size[right[i]]

alive_size = subtree_size[:]
deleted = [False] * (n + 1)

m = int(data[pos])
pos += 1

result = []
remaining = n

for _ in range(m):
    x = int(data[pos])
    pos += 1

    if x not in key_to_vertex:
        result.append(str(remaining))
        continue

    v = key_to_vertex[x]

    current = v
    already_deleted = False

    while current != 0:
        if deleted[current]:
            already_deleted = True
            break
        current = parent[current]

    if already_deleted:
        result.append(str(remaining))
        continue

    removed_count = alive_size[v]
    deleted[v] = True

```

```

        remaining -= removed_count

        current = v

        while current != 0:
            alive_size[current] -= removed_count
            current = parent[current]

        result.append(str(remaining))

    with open("output.txt", "w", encoding="utf-8") as f:
        f.write("\n".join(result) + "\n")

tracemalloc.start()
start_time = time.perf_counter()

solve()

finish_time = time.perf_counter()
cur, peak = tracemalloc.get_traced_memory()
tracemalloc.stop()

with open("stats.txt", "w", encoding="utf-8") as f:
    f.write(f"time = {finish_time - start_time:.6f} sec\n")
    f.write(f"memory = {peak / 1024:.2f} KB\n")

```

Текстовое объяснение решения:

В этой задаче нужно после каждого запроса удалять не одну вершину, а всю вершину вместе с ее поддеревом.

Сначала программа считывает дерево и сохраняет для каждой вершины ее ключ, левого ребенка, правого ребенка и родителя. Также создается словарь `key_to_vertex`, который позволяет быстро найти номер вершины по ее ключу.

Далее для каждой вершины заранее вычисляется размер ее поддерева. Для этого вершины обрабатываются в обратном порядке, так как по условию

номера детей больше номера родителя. Размер поддерева вершины равен единице плюс размеры левого и правого поддеревьев.

Чтобы корректно обрабатывать повторные удаления, используется массив `deleted`. Если вершина уже была удалена сама или вместе с предком, удалять ее повторно не нужно.

Для каждого запроса программа сначала проверяет, существует ли вершина с таким ключом. Если такой вершины нет, количество оставшихся вершин не меняется.

Если вершина есть, программа поднимается от нее к корню и проверяет, не была ли удалена одна из вершин на этом пути. Если была, значит текущая вершина уже удалена вместе с поддеревом предка.

Если вершина еще существует, удаляется всё ее живое поддерево. Количество удаляемых вершин берется из массива `alive_size`. После этого это количество вычитается из общего числа оставшихся вершин и из значений `alive_size` у всех предков.

Так как высота дерева не превосходит 25, подъем к корню выполняется быстро.

Таблица тестирования:

время	память	input	output
0.001876 sec	11.38 KB	6 -2 0 2 8 4 3 9 0 0 3 6 5 6 0 0 0 0 0 4 6 9 7 8	5 4 4 1
0.001665 sec	10.84 KB	1 10 0 0 3 5 20 7	1 1 1

0.002383 sec	10.94 KB	3 2 2 3 1 0 0 3 0 0 3 1 3 5	2 1 1
0.002361 sec	11.28 KB	5 10 2 3 5 4 5 15 0 0 2 0 0 7 0 0 4 5 2 7 15	2 2 2 1

Вывод по задаче

В ходе решения задачи была реализована обработка запросов на удаление поддеревьев в двоичном дереве поиска. Для каждой вершины заранее вычисляется размер поддерева, а для проверки уже удаленных вершин используется подъем к корню. Благодаря ограничению на высоту дерева каждый запрос обрабатывается быстро. Программа корректно выводит количество оставшихся вершин после каждого удаления.

Задача №13. Делаю я левый поворот...

Текст задачи:

Для балансировки AVL-дерева при операциях вставки и удаления используются левые и правые повороты.

Левый поворот выполняется в вершине тогда, когда баланс этой вершины больше 1.

Существует два вида левого поворота:

малый левый поворот;

большой левый поворот.

Если баланс правого ребенка корня не равен -1, выполняется малый левый поворот. Если баланс правого ребенка равен -1, выполняется большой левый поворот.

Дано дерево, в котором баланс корня равен 2. Нужно выполнить левый поворот и вывести получившееся дерево.

Формат ввода:

В первой строке входного файла input.txt находится число N — количество вершин дерева.

В следующих N строках находится описание вершин дерева. В каждой строке записаны три числа:

K_i L_i R_i

где K_i - ключ вершины, L_i - номер левого ребенка, R_i - номер правого ребенка.

Если ребенка нет, вместо его номера указано 0.

Ограничения:

$$1 \leq N \leq 2 * 10^5$$

$$|K_i| \leq 10^9$$

Все ключи различны.

Гарантируется, что данное дерево является деревом поиска.

Гарантируется, что баланс корня равен 2.

Формат вывода:

В выходной файл output.txt нужно вывести дерево после выполнения левого поворота.

В первой строке нужно вывести количество вершин N.

Далее нужно вывести N строк в формате:

Ki Li Ri

Нумерация вершин может быть произвольной, если она соответствует формату.

Код:

```
import time
import tracemalloc

def height(v, left, right):
    if v == 0:
        return 0

    stack = [(v, 0)]
    order = []

    while stack:
        node, used = stack.pop()

        if node == 0:
            continue

        if used == 0:
            stack.append((node, 1))
            stack.append((left[node], 0))
            stack.append((right[node], 0))
        else:
            order.append(node)

    h = {}

    for node in order:
        h[node] = max(h.get(left[node], 0), h.get(right[node], 0))
    + 1

    return h.get(v, 0)
```

```

def get_balance(v, left, right):
    return height(right[v], left, right) - height(left[v], left,
right)

def small_left_rotate(root, left, right):
    new_root = right[root]
    temp = left[new_root]

    left[new_root] = root
    right[root] = temp

    return new_root

def small_right_rotate(root, left, right):
    new_root = left[root]
    temp = right[new_root]

    right[new_root] = root
    left[root] = temp

    return new_root

def solve():
    with open("input.txt", "r", encoding="utf-8") as f:
        data = list(map(int, f.read().split()))

    n = data[0]

    key = [0] * (n + 1)
    left = [0] * (n + 1)
    right = [0] * (n + 1)

    pos = 1

    for i in range(1, n + 1):
        key[i] = data[pos]
        left[i] = data[pos + 1]
        right[i] = data[pos + 2]
        pos += 3

    root = 1
    right_child = right[root]

```



```

if get_balance(right_child, left, right) == -1:
    right[root] = small_right_rotate(right_child, left, right)

new_root = small_left_rotate(root, left, right)

order = []
stack = [new_root]

while stack:
    v = stack.pop()

    if v == 0:
        continue

    order.append(v)

    if right[v] != 0:
        stack.append(right[v])

    if left[v] != 0:
        stack.append(left[v])

new_number = {}

for i, v in enumerate(order, 1):
    new_number[v] = i

result = []

for v in order:
    new_left = 0
    new_right = 0

    if left[v] != 0:
        new_left = new_number[left[v]]

    if right[v] != 0:
        new_right = new_number[right[v]]

    result.append((key[v], new_left, new_right))

with open("output.txt", "w", encoding="utf-8") as f:
    f.write(str(n) + "\n")

    for k, l, r in result:
        f.write(f"{k} {l} {r}\n")

```

```
tracemalloc.start()
start_time = time.perf_counter()

solve()

finish_time = time.perf_counter()
cur, peak = tracemalloc.get_traced_memory()
tracemalloc.stop()

with open("stats.txt", "w", encoding="utf-8") as f:
    f.write(f"time = {finish_time - start_time:.6f} sec\n")
    f.write(f"memory = {peak / 1024:.2f} KB\n")
```

Текстовое объяснение решения:

В этой задаче нужно выполнить левый поворот в корне дерева. При этом необходимо определить, какой именно поворот нужен: малый или большой.

Сначала программа считывает дерево и сохраняет для каждой вершины ее ключ, левого ребенка и правого ребенка.

После этого вычисляется баланс правого ребенка корня. Баланс определяется как разность высот правого и левого поддеревьев.

Если баланс правого ребенка равен -1, то сначала выполняется малый правый поворот в правом ребенке корня. После этого выполняется малый левый поворот в самом корне. Такая комбинация соответствует большому левому повороту.

Если баланс правого ребенка не равен -1, то сразу выполняется малый левый поворот в корне.

После поворота структура дерева меняется. Чтобы вывести дерево в корректном формате, программа заново нумерует вершины. Для этого выполняется обход от нового корня, и каждой вершине присваивается

новый номер. Затем для каждой вершины выводится ее ключ и новые номера детей.

Таблица тестирования:

время	память	input	output
0.002504 sec	10.75 KB	3 1 0 2 2 0 3 3 0 0	3 2 2 3 1 0 0 3 0 0
0.002087 sec	10.75 KB	3 1 0 2 3 3 0 2 0 0	3 2 2 3 1 0 0 3 0 0
0.001732 sec	10.77 KB	4 10 0 2 20 3 4 15 0 0 30 0 0	4 20 2 4 10 0 3 15 0 0 30 0 0
0.002342 sec	10.92 KB	5 10 0 2 30 3 0 20 4 5 15 0 0 25 0 0	5 20 2 4 10 0 3 15 0 0 30 5 0 25 0 0

Вывод по задаче

В ходе решения задачи был реализован левый поворот в корне АВЛ-дерева. Программа определяет, требуется ли малый или большой левый поворот, выполняет необходимые изменения связей между вершинами и заново нумерует дерево для корректного вывода. Решение сохраняет свойство двоичного дерева поиска и выводит получившееся дерево в требуемом формате.

Задача №14. Вставка в AVL-дерево

Текст задачи:

Дано корректное AVL-дерево. Нужно вставить в него новую вершину с ключом X .

Вставка выполняется как в обычном двоичном дереве поиска: сначала находится место для новой вершины, затем вершина добавляется в дерево.

После этого необходимо подняться от добавленной вершины к корню и восстановить баланс дерева. Если какая-то вершина стала несбалансированной, нужно выполнить нужный поворот.

Если дерево до вставки было пустым, новая вершина становится корнем дерева.

Задача из лабораторной работы №2, задача №14, стоимость — 3 балла.

Формат ввода:

В первой строке входного файла `input.txt` находится число N — количество вершин в дереве.

В следующих N строках находится описание вершин дерева. Каждая строка содержит три числа:

K_i L_i R_i

где K_i - ключ вершины, L_i - номер левого ребенка, R_i - номер правого ребенка.

Если ребенка нет, вместо его номера указано 0.

В последней строке находится число X — ключ вершины, которую нужно вставить.

Ограничения:

$$0 \leq N \leq 2 * 10^5$$

$$|K_i| \leq 10^9$$

$$|X| \leq 10^9$$

Все ключи различны.

Гарантируется, что данное дерево является корректным AVL-деревом.

Гарантируется, что вершины с ключом X в дереве нет.

Формат вывода:

В выходной файл output.txt нужно вывести дерево после вставки.

В первой строке нужно вывести количество вершин дерева.

Далее нужно вывести описание всех вершин в формате:

Ki Li Ri

Нумерация вершин может быть произвольной, если она соответствует формату.

Код:

```
import time
import tracemalloc

def solve():
    with open("input.txt", "r", encoding="utf-8") as f:
        data = list(map(int, f.read().split()))

    n = data[0]

    key = [0] * (n + 2)
    left = [0] * (n + 2)
    right = [0] * (n + 2)
```

```

parent = [0] * (n + 2)
height = [0] * (n + 2)

pos = 1

for i in range(1, n + 1):
    key[i] = data[pos]
    left[i] = data[pos + 1]
    right[i] = data[pos + 2]
    pos += 3

    if left[i] != 0:
        parent[left[i]] = i

    if right[i] != 0:
        parent[right[i]] = i

x = data[pos]

if n == 0:
    with open("output.txt", "w", encoding="utf-8") as f:
        f.write("1\n")
        f.write(f"{x} 0 0\n")
    return

for i in range(n, 0, -1):
    height[i] = max(height[left[i]], height[right[i]]) + 1

def update(v):
    if v != 0:
        height[v] = max(height[left[v]], height[right[v]]) + 1

def balance(v):
    return height[right[v]] - height[left[v]]

root = 1

def rotate_left(v):
    nonlocal root

    u = right[v]
    b = left[u]
    p = parent[v]

    left[u] = v
    parent[v] = u

    right[v] = b

```

```

        if b != 0:
            parent[b] = v

        parent[u] = p

        if p == 0:
            root = u
        else:
            if left[p] == v:
                left[p] = u
            else:
                right[p] = u

        update(v)
        update(u)

    return u

def rotate_right(v):
    nonlocal root

    u = left[v]
    b = right[u]
    p = parent[v]

    right[u] = v
    parent[v] = u

    left[v] = b
    if b != 0:
        parent[b] = v

    parent[u] = p

    if p == 0:
        root = u
    else:
        if left[p] == v:
            left[p] = u
        else:
            right[p] = u

    update(v)
    update(u)

    return u

new_node = n + 1

```

```

key[new_node] = x
height[new_node] = 1

v = root

while True:
    if x < key[v]:
        if left[v] == 0:
            left[v] = new_node
            parent[new_node] = v
            break
        else:
            v = left[v]
    else:
        if right[v] == 0:
            right[v] = new_node
            parent[new_node] = v
            break
        else:
            v = right[v]

current = parent[new_node]

while current != 0:
    update(current)

    b = balance(current)

    if b == 2:
        if balance(right[current]) < 0:
            rotate_right(right[current])

        new_root = rotate_left(current)
        current = parent[new_root]

    elif b == -2:
        if balance(left[current]) > 0:
            rotate_left(left[current])

        new_root = rotate_right(current)
        current = parent[new_root]

    else:
        current = parent[current]

order = []
stack = [root]

```



```

while stack:
    v = stack.pop()

    if v == 0:
        continue

    order.append(v)

    if right[v] != 0:
        stack.append(right[v])

    if left[v] != 0:
        stack.append(left[v])

new_number = {}

for i, v in enumerate(order, 1):
    new_number[v] = i

result = []

for v in order:
    new_left = 0
    new_right = 0

    if left[v] != 0:
        new_left = new_number[left[v]]

    if right[v] != 0:
        new_right = new_number[right[v]]

    result.append((key[v], new_left, new_right))

with open("output.txt", "w", encoding="utf-8") as f:
    f.write(str(n + 1) + "\n")

    for k, l, r in result:
        f.write(f"{k} {l} {r}\n")

tracemalloc.start()
start_time = time.perf_counter()

solve()

finish_time = time.perf_counter()
cur, peak = tracemalloc.get_traced_memory()
tracemalloc.stop()

```

```
with open("stats.txt", "w", encoding="utf-8") as f:
    f.write(f"time = {finish_time - start_time:.6f} sec\n")
    f.write(f"memory = {peak / 1024:.2f} KB\n")
```

Текстовое объяснение решения:

В этой задаче необходимо вставить новый ключ в АВЛ-дерево и сохранить его сбалансированность.

Сначала программа считывает дерево и сохраняет для каждой вершины ключ, левого ребенка, правого ребенка и родителя. Также для каждой вершины вычисляется высота поддерева.

Если дерево пустое, новая вершина сразу становится корнем.

Если дерево не пустое, выполняется обычная вставка как в двоичное дерево поиска. Программа спускается от корня: если новый ключ меньше текущего, переход идет влево, иначе вправо. Когда находится пустое место, туда добавляется новая вершина.

После вставки программа поднимается от родителя новой вершины к корню. Для каждой вершины пересчитывается высота и баланс.

Баланс считается как разность высот правого и левого поддерева. Если баланс стал равен 2, значит правое поддерево слишком тяжелое и нужен левый поворот. Если при этом правый ребенок имеет отрицательный баланс, сначала выполняется правый поворот в правом ребенке, а затем левый поворот в текущей вершине.

Если баланс стал равен -2, значит левое поддерево слишком тяжелое и нужен правый поворот. Если при этом левый ребенок имеет

положительный баланс, сначала выполняется левый поворот в левом ребенке, а затем правый поворот в текущей вершине.

После восстановления баланса программа заново нумерует вершины обходом от корня и выводит дерево в требуемом формате.

Таблица тестирования:

время	память	input	output
0.003074 sec	11.83 KB	2 3 0 2 4 0 0 5	3 4 2 3 3 0 0 5 0 0
0.001617 sec	10.62 KB	0 10	1 10 0 0
0.003055 sec	11.99 KB	3 10 2 3 5 0 0 15 0 0 12	4 10 2 3 5 0 0 15 4 0 12 0 0
0.002169 sec	11.93 KB	3 10 2 0 5 3 0 2 0 0 1	4 2 2 3 1 0 0 10 4 0 5 0 0

Вывод по задаче

В ходе решения задачи была реализована вставка нового ключа в AVL-дерево. После обычной вставки как в двоичное дерево поиска программа поднимается к корню, пересчитывает высоты вершин и выполняет необходимые повороты для восстановления баланса. Решение корректно обрабатывает пустое дерево, обычную вставку без поворота, а также случаи, когда после вставки требуется балансировка.

Задача №6. Количество пересадок

Текст задачи:

Дан неориентированный граф с n вершинами и m ребрами, а также две различные вершины u и v .

Нужно найти минимальное количество ребер в пути из вершины u в вершину v .

Если пути между вершинами нет, нужно вывести -1.

Задача относится к лабораторной работе №3 по графам и стоит 1 балл.

Формат ввода:

В первой строке входного файла `input.txt` содержатся два числа n и m — количество вершин и ребер.

В следующих m строках содержатся пары чисел a b , обозначающие неориентированное ребро между вершинами a и b .

В последней строке содержатся две вершины u и v , между которыми нужно найти кратчайший путь.

Ограничения:

$$2 \leq n \leq 10^5$$

$$0 \leq m \leq 10^5$$

$$1 \leq u, v \leq n$$

$$u \neq v$$

Формат вывода:

В выходной файл output.txt нужно вывести минимальное количество ребер в пути из u в v.

Если пути нет, нужно вывести -1.

Код:

```
import time
import tracemalloc
from collections import deque

def solve():
    with open("input.txt", "r", encoding="utf-8") as f:
        data = list(map(int, f.read().split()))

    n = data[0]
    m = data[1]

    graph = [[] for _ in range(n + 1)]

    pos = 2

    for _ in range(m):
        a = data[pos]
        b = data[pos + 1]
        pos += 2

        graph[a].append(b)
        graph[b].append(a)

    start = data[pos]
    finish = data[pos + 1]

    dist = [-1] * (n + 1)
    dist[start] = 0

    q = deque()
    q.append(start)

    while q:
        v = q.popleft()

        for to in graph[v]:
            if dist[to] == -1:
                dist[to] = dist[v] + 1
                q.append(to)
```

```
with open("output.txt", "w", encoding="utf-8") as f:
    f.write(str(dist[finish]) + "\n")

tracemalloc.start()
start_time = time.perf_counter()

solve()

finish_time = time.perf_counter()
cur, peak = tracemalloc.get_traced_memory()
tracemalloc.stop()

with open("stats.txt", "w", encoding="utf-8") as f:
    f.write(f"time = {finish_time - start_time:.6f} sec\n")
    f.write(f"memory = {peak / 1024:.2f} KB\n")
```

Текстовое объяснение решения:

В этой задаче нужно найти кратчайший путь в неориентированном невзвешенном графе.

Так как все ребра имеют одинаковую стоимость, для решения используется поиск в ширину — BFS. Этот алгоритм находит минимальное количество ребер от стартовой вершины до всех остальных вершин.

Сначала граф считывается в виде списка смежности. Так как граф неориентированный, каждое ребро добавляется в обе стороны: из *a* в *b* и из *b* в *a*.

После этого создается массив *dist*, в котором хранится расстояние от стартовой вершины до каждой вершины. Изначально все значения равны -1, что означает, что вершина еще не была посещена. Для стартовой вершины расстояние равно 0.

Далее стартовая вершина добавляется в очередь. Пока очередь не пуста, из нее достается очередная вершина. Для всех ее соседей, которые еще не были посещены, расстояние обновляется как расстояние до текущей вершины плюс один.

После завершения BFS в массиве `dist[finish]` будет находиться длина кратчайшего пути. Если вершина `finish` осталась непосещенной, значение будет равно -1.

Таблица тестирования:

время	память	input	output
0.000863 sec	11.34 KB	4 4 1 2 4 1 2 3 3 1 2 4	2
0.000954 sec	11.38 KB	5 4 5 2 1 3 3 4 1 4 3 5	-1
0.000936 sec	11.42 KB	6 5 1 2 2 3 3 4 4 5 5 6 1 6	5
0.000815 sec	11.29 KB	5 2 1 2 4 5 1 5	-1

Вывод по задаче

В ходе решения задачи был реализован поиск кратчайшего пути в неориентированном невзвешенном графе. Для этого использовался алгоритм BFS, который последовательно обходит вершины по уровням и гарантирует нахождение минимального количества ребер до каждой достижимой вершины. Программа корректно выводит длину кратчайшего пути или -1, если пути между заданными вершинами нет.

Задача №12. Цветной лабиринт

Текст задачи:

В парке организован аттракцион «Цветной лабиринт». Он состоит из n комнат, соединенных m двунаправленными коридорами.

Каждый коридор покрашен в один из s цветов. От каждой комнаты отходит не более одного коридора каждого цвета.

Человек начинает путь в комнате номер 1. Ему дана последовательность цветов $c_1, c_2, \dots, c_k$. Каждый раз он должен смотреть на очередной цвет и идти по коридору такого цвета. Если из текущей комнаты нельзя пойти по коридору нужного цвета, человек остается в той же комнате.

Нужно определить, в какой комнате окажется человек после выполнения всей последовательности цветов.

Формат ввода:

В первой строке входного файла `input.txt` содержатся числа n, m, k — количество комнат, количество коридоров и длина последовательности цветов.

В следующих m строках содержатся три числа:

$u \ v \ c$

где u и v - комнаты, соединенные коридором, а c - цвет коридора.

В последней строке содержатся k чисел — последовательность цветов.

Ограничения:

$$1 \leq n \leq 100$$

$$0 \leq m \leq 10000$$

$$1 \leq k \leq 10000$$

$$1 \leq c \leq 100$$

От каждой комнаты отходит не более одного коридора каждого цвета.

Формат вывода:

В выходной файл `output.txt` нужно вывести номер комнаты, в которой окажется человек после выполнения всей последовательности цветов.

Код:

```
import time
import tracemalloc

def solve():
    with open("input.txt", "r", encoding="utf-8") as f:
        data = list(map(int, f.read().split()))

    n = data[0]
    m = data[1]
    k = data[2]

    graph = [[0] * 101 for _ in range(n + 1)]

    pos = 3

    for _ in range(m):
        a = data[pos]
```

```

        b = data[pos + 1]
        color = data[pos + 2]
        pos += 3

        graph[a][color] = b
        graph[b][color] = a

    current = 1

    for _ in range(k):
        color = data[pos]
        pos += 1

        if graph[current][color] != 0:
            current = graph[current][color]

    with open("output.txt", "w", encoding="utf-8") as f:
        f.write(str(current) + "\n")

tracemalloc.start()
start_time = time.perf_counter()

solve()

finish_time = time.perf_counter()
cur, peak = tracemalloc.get_traced_memory()
tracemalloc.stop()

with open("stats.txt", "w", encoding="utf-8") as f:
    f.write(f"time = {finish_time - start_time:.6f} sec\n")
    f.write(f"memory = {peak / 1024:.2f} KB\n")

```

Текстовое объяснение решения:

В этой задаче необходимо смоделировать движение человека по лабиринту по заданной последовательности цветов.

Так как от каждой комнаты отходит не более одного коридора каждого цвета, для каждой комнаты и каждого цвета можно однозначно хранить следующую комнату. Поэтому используется двумерный массив `graph`, где `graph[v][c]` - это комната, в которую можно перейти из комнаты `v` по коридору цвета `c`.

Сначала программа считывает все коридоры. Так как коридоры двунаправленные, переход записывается в обе стороны: из `a` в `b` и из `b` в `a`.

После этого текущая комната устанавливается равной 1. Затем программа по очереди обрабатывает все цвета из последовательности. Если из текущей комнаты есть коридор нужного цвета, человек переходит в соседнюю комнату. Если такого коридора нет, текущая комната не меняется.

После обработки всей последовательности программа выводит номер комнаты, в которой человек оказался в конце.

Таблица тестирования:

время	память	input	output
0.001083 sec	14.39 KB	4 4 5 1 2 1 2 3 2 3 4 3 1 4 4 1 2 3 4 1	2
0.000783 sec	13.50 KB	3 2 4 1 2 5 2 3 7 5 7 1 7	2
0.000715 sec	12.65 KB	2 0 3 1 2 3	1
0.000958 sec	15.18 KB	5 4 6 1 2 10 2 3 20 3 4 30	5

		4 5 40 10 20 30 40 50 10	
--	--	-----------------------------	--

Вывод по задаче

В ходе решения задачи была реализована симуляция движения по цветному лабиринту. Для быстрого перехода по цвету используется таблица переходов из каждой комнаты. Программа корректно обрабатывает как существующие переходы по заданному цвету, так и ситуации, когда нужного коридора нет и человек остается в текущей комнате.

Задача №18. Построение дорог

Текст задачи:

Даны n точек на плоскости. Нужно соединить некоторые пары точек отрезками так, чтобы между любыми двумя точками существовал путь.

При этом суммарная длина всех выбранных отрезков должна быть минимальной.

Иными словами, нужно построить минимальное остовное дерево для полного графа, где вершинами являются точки, а вес ребра равен евклидовому расстоянию между двумя точками.

Задача относится к лабораторной работе №3 по графам и стоит 4 балла.

Формат ввода:

В первой строке входного файла input.txt содержится число n - количество точек на плоскости.

В следующих n строках содержатся координаты точки:

$x_i \ y_i$

Ограничения:

$$1 \leq n \leq 200$$

$$-10^3 \leq x_i, y_i \leq 10^3$$

Все точки попарно различны.

Никакие три точки не лежат на одной прямой.

Формат вывода:

В выходной файл output.txt нужно вывести минимальную общую длину выбранных отрезков.

Ответ должен отличаться от правильного не более чем на 10^{-6} , поэтому нужно вывести число как минимум с семью знаками после запятой.

Код:

```
import time
import tracemalloc
import math

def solve():
    with open("input.txt", "r", encoding="utf-8") as f:
        data = list(map(int, f.read().split()))

    n = data[0]

    points = []
    pos = 1

    for _ in range(n):
        x = data[pos]
        y = data[pos + 1]
        pos += 2

        points.append((x, y))

    used = [False] * n
    min_dist = [float("inf")] * n
    min_dist[0] = 0.0
```

```

answer = 0.0

for _ in range(n):
    v = -1

    for i in range(n):
        if not used[i] and (v == -1 or min_dist[i] <
min_dist[v]):
            v = i

    used[v] = True
    answer += min_dist[v]

    x1, y1 = points[v]

    for to in range(n):
        if not used[to]:
            x2, y2 = points[to]
            dist = math.sqrt((x1 - x2) ** 2 + (y1 - y2) ** 2)

            if dist < min_dist[to]:
                min_dist[to] = dist

    with open("output.txt", "w", encoding="utf-8") as f:
        f.write(f"{answer:.9f}\n")

tracemalloc.start()
start_time = time.perf_counter()

solve()

finish_time = time.perf_counter()
cur, peak = tracemalloc.get_traced_memory()
tracemalloc.stop()

with open("stats.txt", "w", encoding="utf-8") as f:
    f.write(f"time = {finish_time - start_time:.6f} sec\n")
    f.write(f"memory = {peak / 1024:.2f} KB\n")

```

Текстовое объяснение решения:

В этой задаче нужно соединить все точки отрезками минимальной суммарной длины так, чтобы между любыми двумя точками существовал путь.

Такую задачу можно рассматривать как поиск минимального остовного дерева. Каждая точка является вершиной графа, а между каждой парой точек можно провести ребро. Вес ребра равен расстоянию между точками.

Для решения используется алгоритм Прима. Сначала в остов добавляется первая точка. Для каждой еще не добавленной точки хранится минимальное расстояние до уже построенной части остова.

На каждом шаге выбирается еще не использованная точка с минимальным значением `min_dist`. Это расстояние добавляется к ответу, а сама точка включается в остов.

После добавления новой точки программа пересчитывает расстояния до всех остальных еще не использованных точек. Если расстояние через новую точку меньше уже сохраненного значения, оно обновляется.

Так как количество точек не превышает 200, можно не строить явно все ребра полного графа. Расстояния между точками вычисляются прямо во время работы алгоритма.

После добавления всех точек в остов сумма выбранных расстояний является минимальной общей длиной дорог..

Таблица тестирования:

время	память	input	output
0.001130 sec	10.70 KB	4 0 0 0 1 1 0 1 1	3.000000000

0.000772 sec	10.77 KB	5 0 0 0 2 1 1 3 0 3 2	7.064495102
0.001103 sec	10.47 KB	1 0 0	0.000000000
0.000944 sec	10ю59 KB	3 0 0 3 0 0 4	7.000000000

Вывод по задаче

В ходе решения задачи был реализован алгоритм Прима для построения минимального остовного дерева. Точки рассматриваются как вершины полного графа, а расстояния между ними — как веса ребер. Программа постепенно добавляет в остов ближайшую точку и обновляет минимальные расстояния до остальных точек. Решение корректно находит минимальную суммарную длину дорог и выводит ответ с необходимой точностью.

.

Задача №19. Кластеризация

Текст задачи:

Кластеризация - это задача разбиения набора объектов на группы, которые называются кластерами.

Даны n точек на плоскости и число k . Нужно разбить все точки на k непустых кластеров так, чтобы расстояние между любыми двумя точками из разных кластеров было как можно больше.

Нужно найти максимально возможное значение d , при котором все точки можно разбить на k кластеров так, что расстояние между любыми двумя точками из разных кластеров будет не меньше d .

Задача из лабораторной работы №3 по графам, стоимость — 4 балла.

Формат ввода:

В первой строке входного файла `input.txt` содержится число n - количество точек.

В следующих n строках содержатся координаты точек:

$x_i \ y_i$

В последней строке содержится число k — количество кластеров.

Ограничения:

$$2 \leq k \leq n \leq 200$$

$$-10^3 \leq x_i, y_i \leq 10^3$$

Все точки попарно различны.

Формат вывода:

В выходной файл `output.txt` нужно вывести максимально возможное значение d .

Ответ должен отличаться от правильного не более чем на 10^{-6} , поэтому нужно вывести число с достаточным количеством знаков после запятой.

Код:

```
import time
```

```
import tracemalloc
import math

def find(parent, v):
    if parent[v] != v:
        parent[v] = find(parent, parent[v])
    return parent[v]

def union(parent, rank, a, b):
    a = find(parent, a)
    b = find(parent, b)

    if a == b:
        return False

    if rank[a] < rank[b]:
        a, b = b, a

    parent[b] = a

    if rank[a] == rank[b]:
        rank[a] += 1

    return True

def solve():
    with open("input.txt", "r", encoding="utf-8") as f:
        data = list(map(int, f.read().split()))

    n = data[0]

    points = []
    pos = 1

    for _ in range(n):
        x = data[pos]
        y = data[pos + 1]
        pos += 2

        points.append((x, y))

    k = data[pos]

    edges = []
```

```

for i in range(n):
    x1, y1 = points[i]

    for j in range(i + 1, n):
        x2, y2 = points[j]

        dist = math.sqrt((x1 - x2) ** 2 + (y1 - y2) ** 2)
        edges.append((dist, i, j))

edges.sort()

parent = [i for i in range(n)]
rank = [0] * n

components = n

for dist, a, b in edges:
    if find(parent, a) != find(parent, b):
        if components == k:
            with open("output.txt", "w", encoding="utf-8") as
f:
                f.write(f"{dist:.9f}\n")
            return

        union(parent, rank, a, b)
        components -= 1

tracemalloc.start()
start_time = time.perf_counter()

solve()

finish_time = time.perf_counter()
cur, peak = tracemalloc.get_traced_memory()
tracemalloc.stop()

with open("stats.txt", "w", encoding="utf-8") as f:
    f.write(f"time = {finish_time - start_time:.6f} sec\n")
    f.write(f"memory = {peak / 1024:.2f} KB\n")

```

Текстовое объяснение решения:

В этой задаче точки можно рассматривать как вершины полного графа. Между каждой парой точек есть ребро, вес которого равен евклидову расстоянию между этими точками.

Чтобы получить k кластеров с максимальным расстоянием между разными кластерами, используется идея алгоритма Крускала.

Сначала строятся все ребра между парами точек. Для каждого ребра вычисляется расстояние между двумя точками. Затем все ребра сортируются по возрастанию длины.

Изначально каждая точка является отдельным кластером, то есть всего есть n компонент. Далее программа начинает объединять ближайшие точки так же, как в алгоритме Крускала.

Пока количество компонент больше k , ближайшие разные компоненты объединяются. Когда количество компонент стало равно k , следующее минимальное ребро, которое соединяет две разные компоненты, и будет ответом. Именно оно показывает минимальное расстояние между двумя разными кластерами при оптимальном разбиении.

Для хранения компонент используется система непересекающихся множеств DSU. Она позволяет быстро проверять, находятся ли две точки в одном кластере, и объединять разные кластеры.

Таблица тестирования:

время	память	input	output
0.000915 sec	10.91 KB	4 0 0 0 1 1 0 1 1 2	1.000000000
0.001021 sec	10.90 KB	4 0 0 0 1	1.000000000

		1 0 1 1 4	
0.000849 sec	10.61 KB	3 0 0 3 0 0 4 2	4.000000000
0.000837 sec	11.38 KB	5 0 0 0 2 1 1 3 0 3 2 2	2.236067977

Вывод по задаче

В ходе решения задачи был реализован алгоритм кластеризации точек на плоскости. Для этого был построен полный граф расстояний между точками, после чего использовалась идея алгоритма Крускала. Программа объединяет ближайшие компоненты до тех пор, пока не останется нужное количество кластеров, а затем выводит минимальное расстояние между разными кластерами. Решение корректно находит максимально возможное значение d и выводит его с требуемой точностью..

Вывод по всей работе

В ходе работы были разобраны и реализованы задачи на жадные алгоритмы, двоичные деревья поиска, графы и минимальные остовные деревья. Были использованы сортировка, BFS, DSU, алгоритм Крускала и работа с геометрическими расстояниями. Все решения оформлены в едином стиле, содержат код, объяснение, тестирование и выводы по задачам.